AgentCore as the Sandbox in Agentic RL

AgentCore Runtime as the execution sandbox, integrated with slime

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Outline

1. 1. Why sandboxes — what frontier labs run in them
2. 2. Agentic RL architecture & the training pipeline
3. 3. Foundations of AgentCore Runtime
4. 4. Integrating AgentCore into slime as the sandbox
5. 5. The demos
6. 6. Cross-cutting mechanisms
7. References

Section 1

1. Why sandboxes — what frontier labs run in them

Purpose & roadmap — AgentCore as the sandbox for agentic RL

One idea: in agentic RL the policy acts over many turns against an executable environment, and a **programmatic checker** — not an LLM judge — turns the outcome into a scalar reward (RLVR).

Roadmap (6 sections):

1. **Why sandboxes** — what frontier labs run in sandboxes, and how they score it.
2. **Architecture & pipeline** — reference architecture, the five components, slime + its async rollout.
3. **Foundations of AgentCore** — Runtime / session / microVM, CPU vs GPU compute, session lifecycle & launch/terminate.
4. **Integrating into slime** — the call path, one hook, the episode loop, and the key code.
5. **The demos** — all 9 at a glance, then **one slide per demo** (config, checker, measured eval/latency, honest status, real use case).

Purpose & roadmap — AgentCore as the sandbox for agentic RL (cont.)

6. **Cross-cutting mechanisms** — episode shape, TITO, the scaling `min()` law, one-episode→many-samples, and the resilience levers.

*Scope: AgentCore = the execution **sandbox**; **slime** = the trainer; the **harness stays in-process**.*

Executive summary — the one slide

- **Problem.** Agentic RL runs **untrusted model output** in an isolated environment thousands of times per run and scores it with a **programmatic checker** — the *sandbox* layer, not the GPU, is often the bottleneck (Kimi K2 [5]: **>10,000 concurrent sandboxes**).
- **Solution. AgentCore Runtime is that sandbox** — one isolated microVM session per episode, **in your own AWS account** (IAM/SigV4, VPC), bring-your-own container, CPU by default and **GPU on Instances** — wired into the **slime** trainer through **one hook**, harness in-process.
- **Proven.** The full loop runs **end-to-end on real hardware across 6 demos** (5 main + 1 experimental; code, math, SQL, RAG, tool-use, CUDA kernels), each a standard **10-step run** with real rewards from an in-sandbox checker.
- **How you adopt it.** Build a sandbox image → deploy a Runtime → seed a task → train via one `generate` hook. **CPU↔GPU is one env var.**

What tasks do frontier labs run in sandboxes

Untrusted output runs in an isolated sandbox; a **programmatic verifier** produces the reward. This is the dominant recipe in recent tech reports — Kimi K2 [5] reports **>10,000 concurrent sandboxes** for agentic tool-use / code RL.

Two verifier styles: *verifiable / rule-based* (binary, ungameable) vs *reference-based judge* (open-ended). This project uses **RLVR only**.

Model	Verifier	Tasks	Reward
DeepSeek-R1 [2]	rule verifier + compiler	math, competitive code	tests pass / match
Qwen3 [3]	RLVR + live tool env	math, code, tool-use, RAG	rule + judge
Llama-Nemotron [4]	LLM-judge	science, math, instruction	match + format
Kimi K2 [5]	simulator + real exec sandboxes	agentic tool use, code, SWE	tests + checks

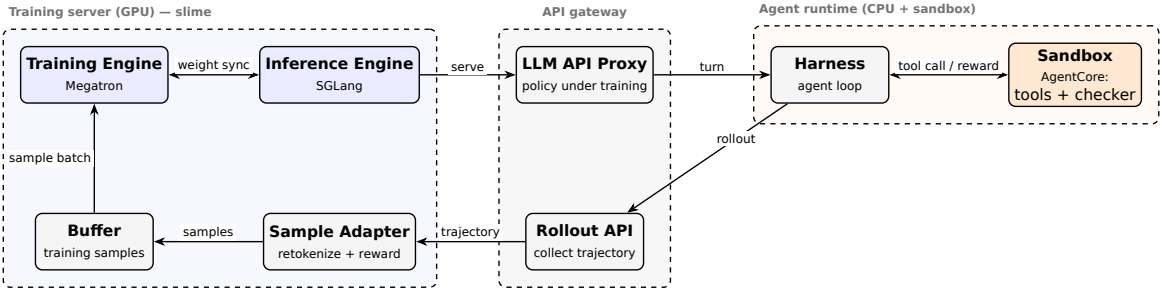
What tasks do frontier labs run in sandboxes (cont.)

Same recipe throughout this deck: ***one isolated sandbox per episode****, a* ***programmatic checker*** *for the reward.*

Section 2

2. Agentic RL architecture & the training pipeline

The reference architecture — one diagram



Generation lane (top, →); training-data lane (bottom, ←); disaggregated design after Agent Lightning [1], slime [9] as trainer. ***AgentCore fills the sandbox seam.***

The pipeline — five components, one job each

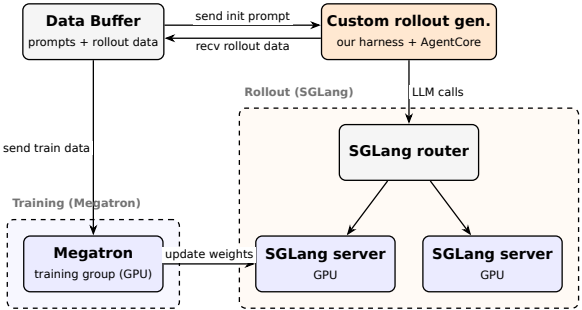
- **Trainer / Inference** (GPU) — learn, and generate turns at the *current* policy (kept fresh by **weight sync**).
- **Proxy** (CPU) — makes every model call hit the *policy under training*, not an external API.
- **Harness** (CPU) — the agent loop that owns one episode end-to-end.
- **Sandbox** (CPU/GPU) — the isolated box untrusted output runs in; the **checker scores the reward**.

Two lanes, one cycle:

- **Generation lane (→):** trainer weights → SGLang → proxy → harness turn → sandbox tool call → observation → next turn.
- **Training-data lane (←):** trajectory → reward attached → loss-masked sample → buffer → trainer step → weight sync.

AgentCore fills the sandbox seam — ad-hoc + isolated per episode. The harness needs *warmth*, not isolation, so it stays **in-process**.

slime — the trainer, and its asynchronous rollout



slime [9] = Megatron (training) + SGLang [27] (rollout), bridged by a **Data Buffer**; the agentic loop plugs into its rollout **generation hook** — no fork of the training kernel.

- **Sync mode** alternates train / rollout — each side's GPUs idle while the other runs.
- **Async driver** (`train_async.py`) overlaps them: Megatron trains batch k while the rollout side already generates batch $k{+}1$.

slime — the trainer, and its asynchronous rollout (cont.)

- **Fully-async rollout** (`fully_async_rollout`) keeps a *fixed pool of in-flight episodes across rollout boundaries*, so a step never waits on the slowest trajectory — the **long-tail** problem of agentic RL.
- **Why it matters here:** AgentCore sandboxes have **variable, sometimes long** per-episode latency; async rollout keeps the trainer fed while they run — at the cost of **staleness** (rollouts generated on an older policy version). This project runs the fully-async path (`MODE=B`).

Section 3

3. Foundations of AgentCore Runtime

AgentCore basics — three concepts, precisely

- **Runtime** — a versioned, containerized host behind an ARN (`CreateAgentRuntime` [D9]). The deployable unit; holds no per-user state.
- **Session** — one episode, addressed by a `runtimeSessionId`. First invoke cold-starts compute; the same id re-routes to that warm compute [D3].
- **microVM** — the unit of isolation: own kernel, CPU, memory, filesystem; sanitized on termination [D2].

Cardinality is 1 : N : 1 — one Runtime serves many sessions; each session = exactly one microVM. **The number of Runtimes tracks distinct sandbox *images*, not tasks.**

Concurrency — episodes, sessions & the Runtime together

`N` concurrent episodes run as `N` session ids on ONE Runtime — not `N` Runtimes.

- The `generate` hook opens **one session per episode** (`CONC` in flight at once); each session id pins that episode to **its own microVM** (or its own GPU instance).
- Every tool op of an episode carries the *same* `runtimeSessionId`, so it returns to the **same warm microVM** — state persists within the episode, isolation holds across episodes.
- The capacity provider places those `N` sessions for you; **one image (one ARN)** backs them all. Add a **second ARN** only for a different image/substrate (e.g. GPU), then round-robin the pool.

So concurrency = more sessions, not more Runtimes.

Why a microVM (not just a container)

Running **untrusted, model-generated code** — isolation is the whole point: one episode's code can't see another's files, the trainer's weights, or the host, and the box is wiped at session end.

	Docker container	**AgentCore microVM**	E2B
Boundary	shared host kernel	**own kernel** (Firecracker-class)	own kernel
Runs your image?	yes	**your OCI image in a microVM**	Dockerfile templates
Managed by	you	**AWS, in your account** (IAM/VPC)	E2B service
GPU	host-dependent	**via Instances**	not core

Two compute types — CPU microVM vs GPU Instances

CPU work belongs on the microVM; Instances exist to add a GPU [D5] (EC2 in your account via a capacity provider). **No cross-session GPU time-slicing** — scale by adding sessions/instances, not by fractioning one (no MIG on L4).

Property	Serverless microVM	Instances (GPU)
GPU	none	g4dn/g5/g6/g6e… + inf2
Max session	8 h	14 d
Isolation	1 microVM / session	1 instance / session (**1 GPU**)
Filesystem	ephemeral	persistent EBS
Agents / session	1	many

Two compute types — CPU microVM vs GPU Instances (cont.)

One session = one whole instance (session→instance is 1:1 — you get the *instance*, not a GPU). Two ids acquire two *separate* instances, so you can't split one multi-GPU box across sessions. To use every GPU of one instance: **one shared session + a self-managed in-instance Ray farm** (`demo_gpu_farm`).

The microVM’s resource envelope — three limits that bite

Serverless isolation is cheap, but the microVM is **small and ephemeral**. Three limits shape every image:

Limit	Value	Why it bites	Work-around
Image size	**≤ ~2 GB** (arm64, project-measured)	a CUDA/torch wheel alone is ~2.7 GB → build fails	install heavy wheels **at `CMD`**, not baked (GPU image: ~90 s cold)
Disk	**ephemeral only** — no persistent volume [D3]	a `git clone`d repo / installed pkg is **wiped at session end**	durable state → S3 / session Memory, or move to **Instances** (EBS) [D5]
vCPU / RAM	**metered, not sized** (vCPU-hr + GB-hr) [D13] — no fixed knob	a data-heavy checker can exceed the account quota	check **Service Quotas**; big fixed footprint → **Instances** (EC2 spec)

The microVM’s resource envelope — three limits that bite (cont.)

- **Example:** the g6 sandbox image can’t bake torch (>2 GB) → `pip install torch` at `CMD` so the wheel lands in the ephemeral workspace at boot; a big SWE repo + persistent workspace → move to **Instances** (EBS, 14 d).
- **Rule of thumb:** *files-in-a-session* for light deps → *CMD-time install* for medium → *Instances* when the footprint is large and **fixed**.

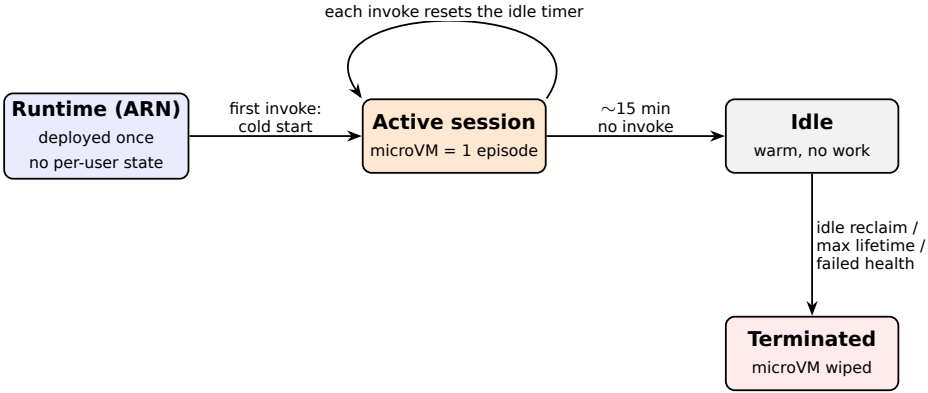
Building & deploying a Runtime — three ways, one contract

Contract [D17]: ARM64 image, serve `0.0.0.0:8080` with `/invocations` + `/ping`. Three ways to put your code on a Runtime (increasing control): the **`@aws/agentcore` CLI** [D16] (zero-Docker zip) → the **`bedrock-agentcore` SDK** [D15] (`@app.entrypoint` implements the contract) → **bring-your-own container** (max control — this project).

```
FROM public.ecr.aws/docker/library/python:3.12-slim   # CPU sandbox
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
CMD ["python","-m","agentcore_slime_demo.sandbox_server_agent"]  # serves :8080
```

- **GPU image:** point `LD_LIBRARY_PATH=/usr/lib64` at the injected `libcuda.so.1`, and `pip install torch` **at `CMD`** (not baked) to stay under the **2048 MB** cap.
- **Deploy:** ECR create → `buildx --platform` (arm64 microVM / amd64 GPU) → push → `CreateAgentRuntime` → (GPU) `CreateCapacityProvider` (`g6.xlarge`, VPC, `idleInstanceTimeout=600`) + attach.

Session lifecycle — launch, idle timer, terminate



*Launch is **implicit** (first invoke); terminate is **implicit** (idle / max-lifetime / health). A heartbeat ping resets the timer (HEARTBEAT=1, any demo).*

How to launch & terminate a session — the mechanics

- **Launch = the first `invoke_agent_runtime` on a fresh `runtimeSessionId`.** There is no explicit "start" call; the id (≥33 chars) *is* the launch. The first op cold-starts the microVM (or GPU instance — minutes on a cold GPU); every later op with that id re-routes to the same warm box.
- **Terminate = passive.** No explicit stop on the happy path — the session ends on **idle reclaim (~15 min)**, **max lifetime** (8 h microVM / 14 d instance), or **failed health**. Each invoke resets the idle timer.
- **Consequence in the RL loop:** a **100-row dataset does not launch 100 instances** — only `CONC` episodes are in flight, recycling through rows over steps. **Concurrency multiplies instances, not data.**
- The **weight-sync pause can trip the idle clock** and silently lose a warm session → the resilience levers in §5.
- **Latency: measure, don't assume** — every op is instrumented (`sandbox_ms`, `sglang_gen_ms`).

Why AgentCore vs. rolling your own sandbox

Every agentic-RL team must solve “run untrusted code, isolated, at scale.” The options and the trade:

	Self-managed K8s / Firecracker	E2B / Modal (SaaS)	**AgentCore Runtime**
Isolation	microVM (you operate)	microVM (their cloud)	**microVM, per session**
In your account / VPC	yes — you build it	**no** — their cloud	**yes** — IAM/SigV4, VPC
Bring-your-own container	yes	limited	**yes** (OCI image)
GPU sandbox	you wire it	limited / none	**yes, on Instances**
Ops burden	**high** (run the fleet)	low	**low** (managed)
Session length	you choose	provider cap	8 h µVM / **14 d Instance**

Why AgentCore vs. rolling your own sandbox (cont.)

The client point: AgentCore gives SaaS-level "no fleet to run" ***without*** *your code or data leaving your account.*

Security & isolation — safe to run untrusted output

Every episode executes model-generated code you did not write. AgentCore makes that safe **by construction**:

- **Per-session microVM isolation** — a hardware-virtualized boundary (not a shared container); one episode's code cannot see another's.
- **In your account, in your VPC** — the sandbox runs under **your IAM**, authenticated with **SigV4**; VPC-mode Runtimes keep traffic on **private endpoints** — **nothing leaves your account**.
- **Bring-your-own image** — you own every dependency, user, and filesystem, and control what the sandbox can reach on the network.
- **Ephemeral by default** — a fresh session per episode, reclaimed on idle; no state bleeds across rollouts unless you deliberately snapshot it (`gpu_single` `SNAPSHOT_EVERY=1`).

This is the enterprise reason to run agentic RL on AWS rather than a third-party sandbox cloud.

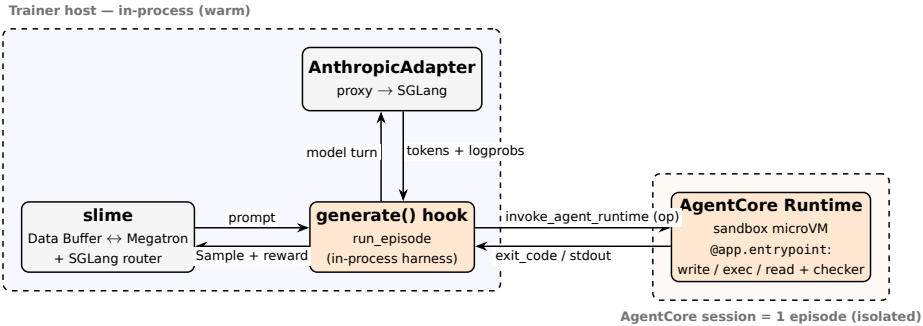
Cost & economics — what you pay, and the packing win

- **Billing model:** you pay for sandbox **vCPU-seconds + GB-hours** while a session is live [D13], plus **Instance** hours for GPU sandboxes — not a flat per-sandbox fee. Idle sessions are reclaimed (~15 min), so **warm-but-unused time is bounded**.
- **CPU sandboxes are cheap; the bottleneck is elsewhere:** for cheap-checker families (math, code, SQL, tool-use) the measured sandbox op is **~0.1 s** (p50 123 ms / p95 900 ms) — generation dominates, so you scale inference, not sandboxes.
- **GPU is where cost concentrates — and where packing pays:** KernelBench is sandbox-bound (exec p50 **8.5 s** ≫ generation 2.5 s). Isolated needs **N whole g6 at once**; the `gpu_farm` Ray farm (`FARM=fractional`) packs **N episodes onto one GPU** (`num_gpus=0.5` → 2×), so you provision **one** instance, not N.
- **Primary lever is capacity, not just cost:** on a capacity-starved GPU fleet you often *cannot* launch N instances at all — the farm makes the run *possible*, and cheaper.

Section 4

4. Integrating AgentCore into slime as the sandbox

How AgentCore sessions integrate into slime — the call path



*One `generate()` hook, one **`runtimeSessionId` per episode**; each tool call = one SigV4 `invoke_agent_runtime`.*

Plugging into slime — one hook, no fork

slime exposes the rollout as a **custom generation function**, so the agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path ⋯generate.generate` — drives the in-process harness; each tool call → one `invoke_agent_runtime` keyed by that episode's **`runtimeSessionId`**.
- **Design choice:** AgentCore is the **sandbox, not the harness**. The harness stays in-process (warm); the sandbox is the isolated per-session microVM. (It *can* host the harness too — `demo_cpu_branching` runs the real Claude Code CLI inside a Runtime.)
- The returned **loss-masked `Sample` + reward** flows straight back into slime's buffer — no fork of Megatron/SGLang.

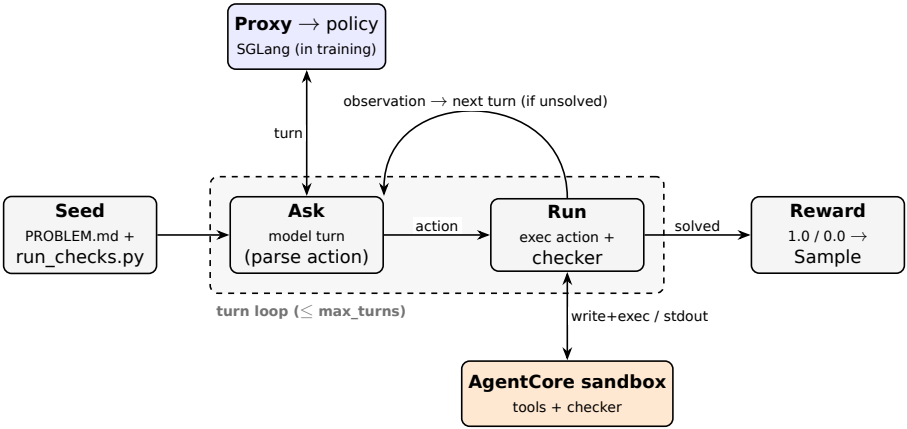
Who owns the lifecycle

slime's training kernel controls **neither** the Runtime nor the session.

- **Runtime** (create/delete the ARN) — the **deploy driver**, once before training. slime just gets the ARN.
- **Session** (per episode) — the `generate` hook's sandbox client: **launch implicit** (first invoke cold-starts the microVM); **terminate implicit** (idle reclaim, no explicit stop on the happy path).

Because termination is *passive*, the weight-sync pause can trip the idle clock and lose a warm session — the failure mode the `HEARTBEAT=1` knob (any demo; was `cpu_heartbeat`) addresses.

The episode, end to end — the loop



Seed → Ask (proxy) → Run (sandbox + checker) → Reward; unsolved loops back.

Step-by-step — one MBPP episode

1. **Deploy (once):** build the arm64 sandbox image → `CreateAgentRuntime` → get the ARN (the deploy driver, not slime).
2. **Launch run:** `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
3. **Open session:** slime hands a prompt to `generate()`, which rebuilds the `Task` and calls `make_sandbox("agentcore")` → **cold-starts** that episode's microVM (`runtimeSessionId`).
4. **Seed:** `write_file` `PROBLEM.md` + hidden `run_checks.py` into the session.
5. **Turn loop (≤ `EPISODE_MAX_TURNS`):** `policy.complete` via the **proxy** (TITO tokens) → extract `solution.py` → `exec("python3 run_checks.py")` in the sandbox → on failure, feed stdout back.
6. **Reward & learn:** `exit_code == 0` → **reward 1.0/0.0** from the in-sandbox checker → `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → **weight sync** to SGLang.

Key code (1/2) — the in-process generate hook

slime calls this once per prompt: build the policy at the SGLang router, open **one sandbox per episode**, run the loop, return one loss-masked `Sample`.

```python
# agentcore_slime_demo/generate.py   (--custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPolicy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_po
                           model_path=args.hf_checkpoint)
    task    = _task_from_sample(sample)            # rebuild Task from row metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"))
    async with sandbox as sb:                      # one session == one episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, logprobs = build_token_stream(traj, tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=logprobs)
```

- Loss mask = **1 on the policy's own tokens, 0 on prompt / observation / tool tokens**.
- `build_token_stream` reuses the **recorded `token_ids` per turn** — no re-tokenization (TITO); returns a *bare* `Sample` (slime's async path chokes on nested lists).

Key code (2/2) — the sandbox client & the Runtime server

Each `write_file` / `exec` / `read_file` = **one SigV4 `invoke_agent_runtime`** [D8] keyed by the episode's session id; boto3 offloaded with `asyncio.to_thread` so N episodes run concurrently (with retries for transient blips + the GPU cold launch).

```python
# client — agentcore_runtime_sandbox.py
def _invoke_sync(self, payload):
    resp = self._client.invoke_agent_runtime(agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,        # >=33 chars; one per episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

# server (deployed image) — the checker runs HERE, so the microVM is the reward source
app = BedrockAgentCoreApp()
@app.entrypoint
def entrypoint(request):        # dispatch on op: ping / write_file / read_file / exec
    return handle(request)
app.run()                       # serves 0.0.0.0:8080  (/invocations + /ping)
```

CPU → GPU is one env var (`SANDBOX=`) — zero change to the loop.

One image, many tasks & dependencies

One microVM = one image per session. Task diversity is absorbed *inside* one generic image, in increasing order of how structural the difference is:

1. **Files, not images** — most CPU families (math `sympy`, code `unittest`, SQL `sqlite3`, tool-use, light CTF) share **one generic Python image**; prompt, seed files, and hidden checker are written into *that episode's session*.
2. **Per-session install** — heavier deps go into the ephemeral workspace: **`CMD`-time** (the GPU image installs the ~2.7 GB torch wheel at boot to stay under the ~2 GB arm64 cap) or **seed-time** `git clone`/`pip`.
3. **Per-capability image** — only a genuine substrate difference (chiefly the **GPU** toolchain: `nvcc` + a real GPU, or a prebuilt dependency set) needs a **second Runtime ARN** (`demo_cpu_tasks VARIANT=imagepool`, the GPU demos).

Section 5

5. The demos

Find your workload — the demos are client use cases

The 6 demos aren't toys; each stands in for a **real agent a client would train**. Find yours:

- **Coding agents** → `cpu_tasks` `DATASET=mbpp` (MBPP repair) / `=humaneval` (function synthesis), `cpu_terminalbench` (whole-OS terminal tasks), `cpu_branching` (a real Claude Code CLI agent; experimental, blocked) — test-driven assistants that iterate against your test runner.
- **Reasoning / math** → `cpu_tasks` `DATASET=math` / `gsm8k` / `math500` / `deepscaler` — reasoning-model post-training with a one-line numeric checker.
- **Data & analytics** → `cpu_tasks` `DATASET=gretelsql` (text-to-SQL), `cpu_tasks` `DATASET=mixed` (SQL + datasci in one stream), `cpu_tasks` `VARIANT=imagepool` (pandas / scipy / sklearn) — agents whose reward is *running the real data stack*.
- **RAG / search** → `cpu_tasks` `DATASET=hotpotqa` (multi-hop) — retrieval agents over your corpus.
- **Tool / API use** → `cpu_tasks` `DATASET=xlam` / `mcp` — function-calling and MCP-schema agents for your APIs.

Find your workload — the demos are client use cases (cont.)

- **GPU kernels / systems** → `gpu_single`, `gpu_farm` — agents whose checker compiles and runs CUDA on a real GPU.

*The rest (`scale_lanes`, `cpu_tasks`' `compare_backends.py` and `VARIANT=bigimage`, `HEARTBEAT=1`, `gpu_single` `SNAPSHOT_EVERY=1`) show **how to scale and operate**; mixing is `cpu_tasks` `DATASET=a,b,c` / `mixed` these in production.*

The demo set — 6 demos, one design

Policy always **Qwen3-4B**, harness **in-process** (except `cpu_branching`), AgentCore the **sandbox**; only the *substrate* and *task* vary — **5 main (3 CPU + 2 GPU) + 1 experimental**. Every demo runs the same six step scripts (`1_prepare_data.sh` … `6_cleanup.sh`):

- **CPU reference** — `cpu_tasks`: ONE demo, many real RLVR datasets via a `DATASET=` knob (mbpp ·math ·gsm8k ·math500 ·deepscaler ·humaneval ·gretelsql · hotpotqa ·xlam ·mcp), a comma list to mix them (`DATASET=gsm8k,gretelsql,hotpotqa`), or `mixed` (math + SQL + CTF + datasci). *A single dataset is the one-element mix. These were ten per-dataset folders + `cpu_mixtask` until 2026-09-23 / 25; they differed only in data, so they were merged. Side notes: `VARIANT=bigimage` (was `cpu_bigimage`) runs the same trainer on a ~4 GB image on a warm CPU Instance; `VARIANT=imagepool` (was `cpu_imagepool`) routes each family to its own image; `compare_backends.py` (was `cpu_customruntime`) compares managed vs your own Runtime.*
- **CPU scale** — `cpu_scale_lanes` (4 × 4 × 4 lanes: harness : engine : microVM; was `cpu_harness_scale`).

The demo set — 6 demos, one design (cont.)

- **CPU image pool at task scale** — `cpu_terminalbench` (one Runtime per task, pre-opened).
- **Every demo** — HEARTBEAT=1 keeps idle sessions warm across the weight sync (was `cpu_heartbeat`, merged 2026-09-25).
- **Experimental** — `cpu_branching` (real Claude Code CLI; **blocked**: the in-sandbox CLI exits 1 before any model call, so no trajectories).
- **GPU** — `gpu_single` (+ custom-Runtime angle and SNAPSHOT_EVERY=1 snapshot/resume; was `gpu_customruntime` / `gpu_snapshot` until 2026-09-25), `gpu_farm` (local Ray farm; FARM=`fractional` packs two episodes on one GPU, FARM=`whole` gives each a whole GPU). *`gpu_farm` replaces `gpu_shared_single`, `gpu_shared_multi` and `gpu_local` (merged 2026-09-25; they differed only in farm knobs).*

The demo set — 6 demos, one design (cont.)

Honest status — uniform. Every training demo is **one 10-step run**: 10 train steps · 1 held-out eval ·1 checkpoint ·a weight-sync each step (10) — **loop-complete, not convergence**. Exceptions: `cpu_branching` is blocked (no trajectories), and `cpu_terminalbench`'s first smoke (2026-09-23) passed but with a sparse reward (4/382 solves). Both GPU demos in every configuration (three `gpu_single` angles, both `gpu_farm` modes) passed the 2026-09-21 GPU sweep. *One slide per demo follows the summary.*

All 6 demos at a glance

Demo	Substrate ·task	Checker	Status / measured
CPU (3 + experimental)			
`cpu_tasks =mbpp`	CPU µVM ·MBPP repair	hidden asserts	10-step; eval 0.56
`cpu_tasks =math`	CPU µVM ·math words	numeric assert	10-step; `eval/gsm8k` 0.94
`cpu_tasks =gsm8k`	CPU µVM ·GSM8K	exact int match	10-step; eval 0.91
`cpu_tasks =math500`	CPU µVM ·MATH-500	numeric 1e-4	10-step; eval 0.83
`cpu_tasks =deepscaler`	CPU µVM ·DeepScaleR	numeric 1e-4	10-step; eval 0.70
`cpu_tasks =humaneval`	CPU µVM ·HumanEval	row `check()` test	10-step; eval 0.81
`cpu_tasks =gretelsql`	CPU µVM ·Gretel SQL	result-set eq (sqlite3)	10-step; eval 0.65
`cpu_tasks =hotpotqa`	CPU µVM ·HotpotQA	SQuAD-norm EM	10-step; eval 0.21
`cpu_tasks =xlam`	CPU µVM ·xLAM-60k	arg-norm call match	10-step; eval 1.00
`cpu_tasks =mcp`	CPU µVM ·ToolACE (MCP)	arg-norm call match	10-step; eval 1.00
`cpu_tasks =mixed`	CPU µVM · math+SQL+CTF+datasci	per-row checker	10-step (as `cpu_mixtask`); `eval/mixed` 0.67
↳ `VARIANT=bigimage`	CPU Instance (m5) · DeepScaleR, ~4 GB image	numeric assert	10-step (as `cpu_bigimage`); eval 0.76
↳ `VARIANT=imagepool`	CPU µVM ×3 · datasci/scipy/sklearn	per-family checker	10-step (as `cpu_imagepool`); eval 1.00
↳ custom Runtime	your CPU Runtime · DeepScaleR	numeric assert	10-step (as `cpu_customruntime`); eval 0.69
↳ `HEARTBEAT=1`	CPU µVM ·DeepScaleR + heartbeat	numeric assert	offline sim + 10-step (as `cpu_heartbeat`); eval 0.71

All 6 demos at a glance (cont.)

`cpu_scale_lanes`	CPU µVM ×4 persistent · DeepScaleR (fixed)	numeric assert	reworked to 4 × 4 × 4 lanes, offline-verified; earlier 10-step eval 0.33 was the pre-rework topology
`cpu_terminalbench`	CPU µVM ×~229 · Terminal-Bench	task's hidden pytest	218/229 Runtimes deployed, 166 usable; smoke passed to ckpt 10 (2026-09-23), reward sparse (4/382), eval 0.0
`cpu_branching` *(exp.)*	CPU CLI Runtime ·SWE + Claude Code CLI	SWE checker	**blocked** — CLI exits 1 pre-model-call; earlier run reward 0
GPU (2)			
`gpu_single`	GPU g6 ·KernelBench	CUDA `allclose`	10-step; eval 0.525
↳ custom-Runtime angle	custom GPU Runtime · KernelBench	CUDA `allclose`	10-step (as `gpu_customruntime`); eval 0.65
↳ `SNAPSHOT_EVERY=1`	whole-GPU farm + snapshot	CUDA `allclose`	10-step (as `gpu_snapshot`); eval 0.625
`gpu_farm =fractional`	1 local GPU, 0.5+0.5 · KernelBench	CUDA `allclose`	10-step; eval 0.775
`gpu_farm =whole`	2 local GPUs, 1 each · KernelBench	CUDA `allclose`	two 10-step runs; eval 0.65 / 0.70

All 6 demos at a glance (cont.)

*Every training demo = one **10-step run** (10 train ·1 eval ·1 ckpt ·10 weight syncs) — loop-complete, not convergence.*

What the 6 demos demonstrate

- **The core claim.** `cpu_tasks` (MBPP by default) and `gpu_single` (KernelBench) are the two reference training loops, on a CPU and a GPU sandbox. *Shows the loop works on both substrates.*
- **Breadth across real workloads.** `cpu_tasks` swaps the dataset with one knob (`DATASET=`): GSM8K, MATH-500, DeepScaleR, HumanEval, Gretel SQL, HotpotQA, xLAM, ToolACE, spanning math, code, SQL, RAG and tool-use. *Shows the same task-agnostic sandbox + checker covers every frontier family with no image change. The dataset is data, not a new demo.*
- **Scale and mix.** `scale_lanes` makes the `min()` throughput law concrete with 4 lanes of harness worker : SGLang engine : microVM, scaled together by `LANES=`; `cpu_tasks DATASET=mixed` (or any comma list) trains one generalist across families in a single stream.

What the 6 demos demonstrate (cont.)

- **Substrate / design variants — the AgentCore mechanics.** `cpu_branching` (real Claude Code CLI as harness; experimental, blocked), `cpu_tasks`' `compare_backends.py` / `gpu_single`'s custom-Runtime angle (managed vs. your own Runtime), `cpu_tasks VARIANT=imagepool` / `cpu_terminalbench` (image pools routed by family / by task), `cpu_tasks VARIANT=bigimage` (image past the microVM cap, on an Instance), `gpu_farm` (GPU Ray farm, fractional or whole-GPU). *Shows the substrate choices a production deployment actually makes.*
- **Resilience.** `HEARTBEAT=1` (any demo) + `gpu_single SNAPSHOT_EVERY=1`: *heartbeat prevents loss; snapshot recovers from it.*

cpu_tasks — one demo, many datasets (DATASET=)

One folder, one harness, one CPU sandbox; the dataset is a knob. DATASET=<name> picks the seed rows + hidden checker and writes eval/<name> + ckpt/Qwen3-4B_<name>_async. An unknown name fails fast. **Why one demo:** the ten former per-dataset folders (demo_cpu_mbpp, demo_cpu_math, demo_cpu_gsm8k ... demo_cpu_mcp) differed only in their data; they were merged on 2026-09-23.

- The next slides give each dataset's measured 10-step run. Those runs were recorded under the old per-dataset folder names.

cpu_tasks DATASET=mbpp — MBPP code repair ·CPU microVM ·10-step run

Substrate CPU µVM ·**Data** MBPP 500/8 ·**Checker** hidden asserts, 1.0 iff exit 0 ·**Use case** unit-test-driven coding assistant.

- **Measured (10-step run):** raw_reward 0.5–0.94, eval/mbpp 0.56 @ step 9; step_time 23–78 s, wait_time_ratio 0.39–0.84, MFU 0.018–0.043 (2–4 % of L40S peak).
- **Latency:** sandbox p50 123 ms / p95 900 ms; SGLang decode p50 1.76 s / p95 4.1 s.
- **Bottleneck:** generation (1.76 s) ≫ sandbox (0.12 s) → **generation-bound**, scale inference. The default DATASET of the reference CPU demo.

cpu_tasks DATASET=math — math word problems ·CPU microVM ·10-step run

Substrate CPU µVM ·**Data** GSM8K 200/40, MATH-500 160/40, DeepScaleR 400/40 (real HF) ·**Checker** numeric assert, 1.0/0.0 ·**Use case** reasoning-model math post-training.

- **Measured (10-step run):** `raw_reward` 0.94→1.0; `eval/gsm8k` 0.94 — a *sanity signal* the loop + numeric checker work, not a curve.
- **Bottleneck:** long reasoning traces dominate, checker is milliseconds → **inference-bound**; `[sglang-gen]`, not `[sbx-op]`, sets step time.
- **To convergence:** NR≈150–200/split, size `--rollout-max-response-len` to the trace p95, scale SGLang.

gpu_single — KernelBench CUDA ·GPU g6 Runtime ·10-step run

Substrate remote g6 (L4) custom Runtime; trainer keeps all 8 L40S ·**Data** KernelBench 100 L1 / 20 L2 ·**Checker** rebuild ref on CUDA, matched seeds, `allclose` + speedup ·**Use case** CUDA kernel-optimization agent.

- **Measured (10-step run):** `raw_reward` 0.28–0.94, `eval/kernelbench` 0.65 (csv)/0.72 (log); `step_time` 110–804 s, `wait_time_ratio` 0.79–0.96 (mostly waiting on the remote checker).
- **Latency (remote g6):** exec p50 8.5 s / p95 17.3 s; first `ping` ~123 s (one-time g6 cold launch); SGLang decode p50 2.5 s / p95 6.4 s.
- **Bottleneck:** exec (8.5 s) ≫ generation (2.5 s) → **sandbox-bound**, scale the sandbox (more g6, or the farm). Re-verified on hardware via `LOCAL_GPU` (`Job succeeded`, ckpt 2).

gpu_farm — a local Ray GPU farm, FARM=fractional | whole ·10-step runs (local GPU)

One demo, two GPU splits (merged 2026-09-25 from gpu_shared_single, gpu_shared_multi, gpu_local): FARM=fractional = two 0.5 actors share ONE GPU; FARM=whole = one whole GPU per episode across TWO GPUs. **Diff from gpu_single:** one line — SANDBOX=agentcore-ray-farm. All episodes invoke **one** GPU Runtime under a shared runtimeSessionId; inside, _handle_farm leases one Ray actor per episode (per-episode sub_session). **Use case** GPU packing on a capacity-starved fleet.

- **Measured (2026-09-21 GPU sweep, 10-step runs):** fractional Job succeeded, ckpt 10, eval/kernelbench **0.775**; whole passed twice, eval 0.65 / 0.70. The client-side open-concurrency gate caps opens at the farm's 2 slots; sandbox open==close counts balanced.

gpu_farm — a local Ray GPU farm, FARM=fractional | whole ·10-step runs (local GPU) (cont.)

- **Why:** isolated needs *N whole g6 at once* (stalls on a starved fleet); the farm needs **one** instance. No MIG on L4 → software-only isolation (`nvcc flock`); `whole` sidesteps the fractional compile-wedge when whole GPUs are available. No convergence curve yet.

HEARTBEAT=1 — heartbeat keeps the sandbox warm ·any demo

Was `cpu_heartbeat`; merged 2026-09-25 into an opt-in knob of every demo (offline probe `demo_cpu_tasks/heartbeat_demo.py`; `cpu_terminalbench` already runs its own keeper). **A mechanism first.** During slime's weight-sync pause generation stalls and an idle session is reclaimed (~15 min); a `SessionKeeper` heartbeat resets the idle clock so the *same* warm microVM survives. **Use case** any long RL run whose warm, stateful session must outlive the sync pause.

- **Three acts:** (1) naive "stop invoking" → reclaimed; (2) heartbeat holds it warm across an arbitrary pause; (3) recover-by-reseed if it dies anyway.
- Lives **outside** the harness by design; the probe runs offline in seconds (no policy). Smoke (as `cpu_heartbeat`, `DATASET=deepscaler HEARTBEAT=1`) `eval/deepscaler` 0.71 with the keeper live. Paired with `gpu_single` `SNAPSHOT_EVERY=1`: *heartbeat prevents loss; snapshot recovers from it.*

gpu_single SNAPSHOT_EVERY=1 — application-level snapshot / resume

Was gpu_snapshot; merged into gpu_single 2026-09-25 (default 0 = off). **A mechanism first.** A GPU instance can be recycled and come back blank (AgentCore has no VM/EBS snapshot), so workspace state is serialized to a store you own (S3 in prod) and restored onto a fresh session. **Use case** surviving an instance recycle mid-episode in a long GPU run.

- **A/B arc:** Run A (no snapshot) redoes from step 0; Run B restores and prints the rework saved.
- **Status:** the **offline sim** is committed (~1 s, `LocalSnapshotStore`). The 2026-09-21 GPU sweep also ran it as a real trainer on the local whole-GPU farm, saving and restoring 3 files per episode: `Job succeeded`, ckpt 10, `eval/kernelbench` 0.625.

cpu_tasks DATASET=gsm8k — grade-school math ·10-step run

Family math ·**Data** GSM8K (openai/gsm8k) 200/40 ·**Checker** exact integer match, 1.0/0.0 ·**Use case** grade-school word-problem solver.

- **Measured (10-step run):** eval/gsm8k **0.91** — the base policy already solves most of this easy split, so it is the *sanity anchor*; its curve should flatten fast.
- **Bottleneck:** generation-bound (cheap checker) → scale inference, never sandboxes.

cpu_tasks DATASET=math500 — competition-style math · 10-step run

Family math ·**Data** MATH-500 (HuggingFaceH4/MATH-500) 160/40, EPISODE_MAX_TURNS=4 ·**Checker** numeric match (tol 1e-4) ·**Use case** competition-math solver.

- **Measured (10-step run):** eval/math500 **0.83** — harder than GSM8K, so more headroom to climb.
- **Note:** its longer reasoning traces make --rollout-max-response-len the first knob to size up. Generation-bound.

cpu_tasks DATASET=deepscaler — RL-scale math (largest split) ·10-step run

Family math ·**Data** DeepScaleR (agentica-org/DeepScaleR-Preview-Dataset) 400/40 — the largest split ·**Checker** numeric match (tol 1e-4), EPISODE_MAX_TURNS=4 · **Use case** RL-scale math post-training.

- **Measured (10-step run):** eval/deepscaler **0.70** — lowest of the math three (hardest cold), the most room to show learning.
- Also the fixed workload cpu_scale_lanes reuses. Generation-bound.

cpu_tasks DATASET=humaneval — function-synthesis coding ·10-step run

Family coding ·**Data** HumanEval (openai/openai_humaneval) 140/24 ·**Checker** the row's own check() unit test, run in the sandbox ·**Use case** function-synthesis coding assistant.

- **Measured (10-step run):** eval/humaneval **0.81** — same shape as DATASET=mbpp but with the canonical HumanEval tests (real code execution, not a numeric assert).
- Generation-bound.

cpu_tasks DATASET=gretelsql — text-to-SQL ·10-step run

Family sql ·**Data** Gretel synthetic text-to-SQL (`gretelai/synthetic_text_to_sql`) 300/40 ·**Checker** result-set equality — query executed against **`sqlite3`** (stdlib), rows vs gold ·**Use case** text-to-SQL analytics agent.

- **Measured (10-step run):** `eval/gretelsql` **0.65**.
- **Cheapest checker of all** (in-process SQLite query) → textbook generation-bound.

cpu_tasks DATASET=hotpotqa — multi-hop RAG ·10-step run

Family rag ·**Data** HotpotQA (`hotpotqa/hotpot_qa`, distractor; 10 passages/question) 300/40 ·**Checker** SQuAD-normalized exact match ·**Use case** multi-hop retrieval / search agent.

- **Measured (10-step run):** `eval/hotpotqa` **0.21** — the **hardest cold** of the datasets, so the most interesting convergence target.
- **Bottleneck:** the mild exception — 10-passage observations grow the *context* each turn, but the checker is trivial → generation/**context**-bound, still not sandbox-bound.

cpu_tasks DATASET=xlam — function-calling / tool use · 10-step run

Family tool use ·**Data** xLAM-60k (`minpeter/xlam-function-calling-60k-parsed`) 300/40 ·**Checker** arg-normalized gold-call match (order-insensitive), RLVR ·**Use case** function-calling tool agent.

- **Measured (10-step run):** `eval/xlam` **1.00** — the base policy nails this slice, so the value is verifying the **tool-call parsing/checker path** end-to-end, not showing a climb.
- Generation-bound.

cpu_tasks DATASET=mcp — MCP-style tool schemas ·10-step run

Family tool use ·**Data** ToolACE (Team-ACE/ToolACE, MCP-shaped schemas) 300/40 · **Checker** arg-normalized gold-call match, RLVR ·**Use case** MCP tool-schema agent.

- **Measured (10-step run, eval-interval 2):** eval/mcp **1.00** on the two-rollout held-out set — a *baseline solve rate*: read it as "the MCP-style tool loop + checker run end-to-end," not a learned gain.
- Same family as xlam, against MCP-shaped tool definitions. Generation-bound.

cpu_tasks DATASET=mixed — one interleaved generalist stream ·10-step run

Substrate CPU µVM, MODE B fully-async ·**Data** `build_mixed_datasets.py` interleaves **math + SQL + CTF + datasci** into one 174-row `mixed_train.jsonl` (datasci rows route to a pandas image via `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`); `DATASET=a,b,c` interleaves any existing datasets round-robin (`eval/mix`) ·**Checker** each row is self-describing (its own family + hidden checker) ·**Use case** post-training **one generalist policy** across skills every batch (the frontier norm).

- **Why nothing changes:** the sandbox is task-agnostic and each row rebuilds its own `EpisodicTask`; GRPO normalizes advantage **within a same-prompt group**, so families' reward scales never clash. Still **one Runtime, many sessions**.
- **Measured (10-step run, as the former `cpu_mixtask`):** `Job succeeded` at ckpt 10, **`eval/mixed` 0.67** (single aggregate metric; per-family breakdown is the open work). Generation-bound.

cpu_scale_lanes — 4 × 4 × 4 lanes: harness : engine : microVM ·offline-verified

Fixes the task (DeepScaleR, 400/40), scales the rollout as lanes. Each lane = one warm harness worker, pinned to **its own SGLang engine** (TP1, one of rollout GPUs 4–7) and **its own persistent AgentCore microVM** (one session for the whole run, a fresh workdir per episode). Default LANES=4 → **4 workers : 4 engines : 4 microVMs**, strictly 1:1:1, on 4 train + 4 inference GPUs; LANES=N scales all three tiers together. *Formerly cpu_harness_scale.* **Use case** seeing which of the three dials caps throughput.

- **Verifiable in logs:** every episode's [demo] line carries lane=, engine=, sandbox_session=; metrics.py checks each lane used one engine and one session.
- **Offline sweep (illustrates the min() law):** throughput 5.3 / 10.5 / 20.9 / 29.6 / 36.5 ep/s at W=1/2/4/6/8 while **harness-bound**, then **plateaus ~37 ep/s** once **inference-bound**. 48/48 solved. Unseeded sim — shape, not absolute rate.

cpu_scale_lanes — 4 × 4 × 4 lanes: harness : engine : microVM ·offline-verified (cont.)

- **Status:** the lane rework is verified offline (fake-microVM run: 4 sessions, 4 episodes per lane, 1:1:1). The earlier hardware run (Job succeeded, ckpt 10, eval/deepscaler 0.33) used the **pre-rework** topology (2 engines, fresh sandbox per episode).

cpu_branching — real Claude Code CLI as harness · EXPERIMENTAL (blocked)

The only demo where the harness is the real Claude Code CLI, co-located with slime; its **tools run in a remote CPU Runtime** (image bakes node22 + `claude`). One episode fans out (subagents + context summarization) into a `list[Sample]`. **Use case** RL on a real production coding agent whose episodes spawn subagents and compact context.

- **Reconciliation:** all leaves tagged with the shared `rollout_id` ρ; advantage baseline + token-mean loss normalized **per rollout, not per sample**, so an incidental fan-out can't reweight the update. `MODE=A`, dp=1 (TP2×PP2), sync save.
- **Measured (10-step run):** `Job succeeded` at ckpt 10; **`fan_out=0` blocker resolved** (=1 ×315, =2 ×8, =3 ×1) — but `raw_reward` and `eval/swe` are still **0.0** (the SWE checker earns no reward yet). **Now blocked:** the in-sandbox CLI exits 1 before any model call, so no trajectories are produced.

cpu_tasks + compare_backends.py — managed CI vs. your own Runtime ·10-step run

Both ways to get a CPU sandbox expose the *identical* `write_file/exec/read_file` contract: the **managed Code Interpreter** (`SANDBOX=agentcore`, zero-ops) vs **your own custom Runtime** (`SANDBOX=agentcore-runtime`, your OCI image, VPC/IAM posture). **Use case** deciding whether a rollout needs a custom image at all.

- *Was `cpu_customruntime`; merged into `cpu_tasks` 2026-09-25.* `compare_backends.py` runs the same episode on both — **reward identical, only operational trade-offs differ**; `DATASET=deepscaler` with your Runtime's ARN is a real training run on it.
- **Measured (10-step run, as `cpu_customruntime`):** `Job succeeded` at ckpt 10, **`eval/deepscaler` 0.69** (per-rollout `raw_reward` 0.72–0.91).

cpu_tasks VARIANT=imagepool — capability-routed image pool ·10-step run

One prebuilt image per capability, each its own Runtime ARN — datasci→pandas, scipy→scipy, sklearn→sklearn — and the harness **routes each episode to the image its family needs**. **Use case** a portfolio whose families pull heavy, incompatible toolkits that can't co-install under the ~2 GB cap.

- Routing is not cosmetic: send a scipy episode to the pandas image and import scipy fails → reward 0. The third rung of the dependency ladder (files → CMD-install → **per-capability ARN**).
- *Was cpu_imagepool; merged into cpu_tasks 2026-09-25 (DATASET=imagepool, now eval/imagepool).*
- **Measured (10-step run, as cpu_imagepool):** Job succeeded at ckpt 10 driving the **real 3-image ARN pool** (per-family routing confirmed in the runtime-env), held-out eval **1.00**.

cpu_terminalbench — one image per task, one fresh microVM per episode ·smoke

Terminal-Bench as a large image pool: one AgentCore Runtime **per task** (~229 images); each episode is routed to its task's own Runtime. It is the `VARIANT=imagepool` routing idea at task scale. Tasks act on the whole OS, so **every episode gets a brand-new session** (isolation). **Use case** terminal / DevOps agents.

- **Cold start hidden by look-ahead:** a slime data-source plugin (`tb_lookahead_source.py`) replays slime's deterministic data order and announces the **next 2 rollouts'** episodes; the pool keeps **64** fresh sessions (2 × 4 prompts × 8 samples) booted ahead, pooled per task, heartbeated until claimed. Offline-tested.
- **Reward:** the task's own hidden pytest oracle (`grade.sh`), RLVR. Tool-calling `MiniClaudeHarness` loop (needs the qwen25 tool parser). Remote-only.
- **Status:** 218/229 Runtimes deployed; oracle validation (reference passes, untouched env fails) **166 usable of 218**; first smoke passed to ckpt 10 on 2026-09-23 (reward sparse: 4 solves / 382 episodes).

Terminal-Bench look-ahead pre-open (1/2): status and why

Status. The look-ahead ran in the 2026-09-23 AWS smoke: `open_wait_ms` p95 was 0 (boot fully hidden), but pre-opened sessions idled for a median of 7.3 min against a 21 s episode, so ~95 % of microVM time was idle (`TB_PREOPEN_BATCHES=1` is the next try). With the defaults (4 prompts × 8 samples, 2 batches ahead), the pool keeps **64 microVMs booted ahead** of the episodes that are running.

How the pool learns the next two batches' episodes. slime would never tell it in time: it passes along only **8 episodes** of the current batch at once and doesn't draw the next batch until the current one finishes. But slime's prompt order is **predictable**: it walks a shuffled dataset in order, with a fixed seed for each epoch. So a data source can compute what slime will draw next.

Terminal-Bench look-ahead pre-open (2/2): the five steps

1. A new **look-ahead data source** (`agentcore_slime_demo/tb_lookahead_source.py`, wired in `run.sh` with `--data-source-path`) wraps slime's normal one. Each time slime draws a batch, it also works out the prompts of the next 2 batches the same way slime will draw them, without moving slime's position.
2. It sends the pool the task list for each upcoming episode: the **32** just drawn, plus the next **64**.
3. The pool keeps the first **64** not-yet-started episodes booted, each on its own brand-new session on that task's Runtime. Sessions are matched by task, and each one still serves exactly one episode.
4. When a worker starts an episode, it takes a ready session for that task and the next episode in line starts booting. So 64 always stay booted ahead, which covers batch **N+2** before slime draws it. The existing heartbeat keeps waiting sessions alive.
5. If a prediction is wrong, the extra sessions are stopped at the next draw and those episodes just open on demand.

cpu_tasks VARIANT=bigimage — an image past the microVM cap, on a warm CPU Instance ·10-step run

Side note on the tasks track: was cpu_bigimage, folded into cpu_tasks 2026-09-25. **A ~4 GB image** (torch + scientific stack) is over the ~2 GB microVM cap, so it runs on a CPU **capacity-provider Instance** (`m5.xlarge`) instead. **Use case** agents whose toolchain can't shrink below the cap.

- **Measured (10-step run, real AWS):** `Job succeeded`, `eval/deepscaler` 0.76. Warm-instance reuse (one shared session, fresh env per episode) cut m5 cold boots from 201 to 1.

gpu_single custom-Runtime angle — a GPU under the sandbox ·capability probe

Was gpu_customruntime; merged into gpu_single 2026-09-25 (gpu_torch_smoke.py; LOCAL_GPU=1 = on-box farm, sync save). **The managed Code Interpreter has no GPU.** If a checker needs CUDA, the sandbox must be **your own custom Runtime on an AgentCore capacity-provider Instance** (a g6 box). The demo write_files a torch-on-CUDA probe, execs it, read_files the result. **Use case** confirming a CUDA-dependent reward can only be scored on the Instances substrate.

- **Measured (probe):** torch.cuda.is_available() **False on managed CI** vs **True on the custom Runtime** (torch 2.11+cu129, device: NVIDIA L40S).
- **Status:** the 2026-09-21 GPU sweep ran it as a real KernelBench trainer: Job succeeded, ckpt 10, eval/kernelbench 0.65 (an earlier LOCAL_GPU attempt was Killed). States the rule: one session = one whole Instance, no cross-session GPU time-slicing.

Section 6

6. Cross-cutting mechanisms

Episode shape decides what you scale

The task's shape — not the domain — sets the config. **One microVM serves many families**: the sandbox is task-agnostic (`write_file` / `exec` / `read_file`); task specificity is *files seeded per episode*, not a new image.

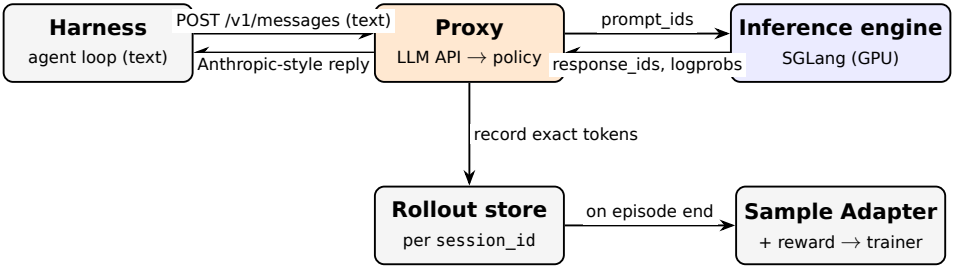
Regime	Example	Bottleneck	Scale
single-turn, long trace	math, reasoning	**generation**	inference
few-turn, heavy checker	KernelBench (CUDA)	**sandbox**	sandbox / GPU
multi-turn, big observations	SWE, RAG, web	context growth	both + warm session

Episode shape decides what you scale (cont.)

- **`--rollout-max-response-len`** — per-turn generation budget; set from the family's p95 per-turn response (math 1k–32k; code 100–800; CUDA kernel 200–1k; tool call 50–300).
- **`EPISODE_MAX_TURNS`** (default 3) — the single biggest lever on episode cost: single-turn → 1; iterative code/SQL → 3–5; long-horizon SWE/web → 15–50+.

Mechanism spotlight — TITO & why the proxy exists

A harness works in **text**; RL trains on **token IDs**. Re-tokenizing decoded text can drift from what the model actually sampled — corrupting the action, the importance ratio, and the loss mask.



Token-in-token-out (TITO): the proxy records the **exact sampled `token_ids` + logprobs** at generation time and forwards them intact, so the trainer learns from the tokens that were really sampled.

Mechanism spotlight — TITO & why the proxy exists (cont.)

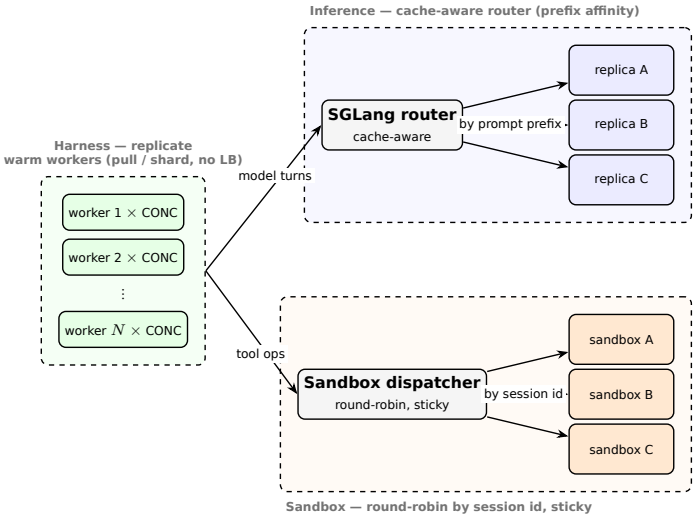
- The proxy is the only component straddling both sides (text ↔ tokens), so it is the only correct place to capture them.
- For binary RLVR rewards this token-level exactness is not optional — slop silently corrupts the signal.

The scaling demo (1/2) — the concrete setup & the three tiers

On the 8×L40S box the demos **fix trainer, inference, and harness** and change only the sandbox ARN + task — the reference-architecture disaggregation made concrete:

Component	Where	Config (shared)
Trainer	4 GPUs	Megatron, GRPO, `global_batch_size=32`, `ACTOR_GPUS=4`
Inference	4 GPUs	SGLang, 2 engines, one **cache-aware router**, `ROLLOUT_GPUS=4`
Harness	in-process (CPU)	`run_episode`, fully-async `MODE=B`, `CONC=4`, `EPISODE_MAX_TURNS=3`
Sandbox	**remote AgentCore**	CPU microVM (arm64) **or** GPU g6 (amd64); one session/episode

The scaling demo (1/2) — the concrete setup & the three tiers (cont.)



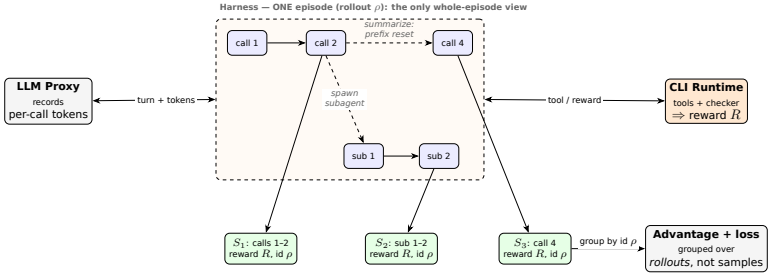
The scaling demo (2/2) — the min() law & `cpu_scale_lanes`

The min() law — `throughput = min(inference, sandbox, harness).` Scale two and the bottleneck moves to the third.

- **Diagnose first:** compare `perf/sglang_gen_ms_*` vs `perf/sandbox_ms_*`. Generation dominates → add SGLang replicas; sandbox dominates → add sessions / ARNs / the GPU farm.
- **`cpu_scale_lanes`** makes all three dials visible as **4 lanes** (worker : engine : microVM, 1:1:1), including the **easily-forgotten third**: the warm in-process harness (GIL, `to_thread` pool, FDs). Scale it by **replicating warm workers**, not by making it ad-hoc. **Problem targeted:** rollout driver becomes the bottleneck once inference and sandbox are both fast.
- **Load-balancing differs by side:** inference → target slime's cache-aware router (prefix affinity); sandbox → the **session id *is* the balancer** (sticky round-robin across a pool of ARNs). You never stand up your own load balancer.

Mechanism spotlight — one episode → many samples

The reconciliation `cpu_branching` implements, and the reason a **real** agent needs it. A production agent (Claude Code CLI) **fans one episode out into many samples** — it spawns subagents (the Task tool) and summarizes context (resetting the token prefix). Scored naively per-sample, that fan-out silently reweights the GRPO baseline and the loss [1].



Mechanism spotlight — one episode → many samples (cont.)

Solution: the CLI runs **inside a CLI-capable CPU Runtime** (image bakes node22 + `claude`); one episode returns a `list[Sample]`, all tagged with the shared `rollout_id` ρ and the episode's terminal reward R. **Advantage baseline + loss are normalized per rollout, not per sample** (token-mean loss), so an incidental fan-out can't reweight the update. `MODE=A`, `dp=1` (TP2×PP2), sync save.

The `cpu_branching` demo (§5) now trains to ckpt 10 with `fan_out` ≥ 1 — the mechanism runs end-to-end; its SWE reward is still 0.

Mechanism spotlight — resilience: heartbeat & snapshot

Two levers a long RL run composes, demonstrated by `HEARTBEAT=1` and `gpu_single SNAPSHOT_EVERY=1` (§5). **Use case:** any multi-hour run whose warm, stateful sessions (installed deps, seeded repo) must survive both the periodic pause and a genuine instance recycle.

- **Heartbeat (`HEARTBEAT=1`, any demo)** — during weight sync generation stalls; an idle session is **reclaimed (~15 min)** [D3] and the next round pays a cold start. A `SessionKeeper` pings on a timer, **resetting the idle clock**, so the *same* warm microVM survives. Lives **outside the harness** (`KeptSandbox`), because the stall is a training-loop property, not an episode one.
- **Snapshot (`gpu_single SNAPSHOT_EVERY=1`)** — a recycled GPU instance comes back blank (no VM/EBS snapshot), so the workspace is serialized to a store you own (S3) and restored onto a fresh session.
- **The pairing:** *heartbeat prevents loss; snapshot recovers from it.*

Adoption path & limits — from pilot to production

Day-1 pilot (a few hours):

1. Build a sandbox image (`Dockerfile.sandbox.cpu`, arm64) and push it; deploy a Runtime (`CreateAgentRuntime` [D9]).
2. Point slime at it with **one env var** (`SANDBOX=agentcore-runtime`) + the `--custom-generate-function-path` hook — **no fork of slime**.
3. Seed a task family, run the standard 10-step loop, watch `eval/<task>` and the `[sbx-op]` / `[sglang-gen]` latencies.
4. Flip **CPU↔GPU** with one env var when your checker needs CUDA.

Limits & quotas to plan around:

- **Session length:** 8 h on a CPU microVM, 14 d on a GPU Instance; **idle ~15 min → reclaimed** (use `HEARTBEAT=1` to survive the weight-sync pause).
- **Image size:** ~2 GB arm64 cap — install heavy deps at `CMD`-time, or route a per-capability image (`cpu_tasks VARIANT=imagepool`).
- **GPU sharing: no MIG** on L4 / L40S — cross-session GPU sharing is software-only (the Ray farm), not hardware-partitioned.

Adoption path & limits — from pilot to production (cont.)

- **Next step after training:** serve the resulting policy on Bedrock / SageMaker — the trained checkpoint is a standard model artifact.

Summary

- **Can:** isolated per-session sandbox for untrusted output (CPU default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, **GPU on Instances**; a one-hook slime integration with the harness in-process.
- **Cannot:** a microVM has no GPU; no cross-session GPU fractioning (no MIG on L4); a session can't exceed 8 h without moving to Instances (14 d).
- **How to run:** build a sandbox image → deploy a Runtime (+ GPU capacity provider) → seed a task → `invoke_agent_runtime` per tool call → in-sandbox checker gives the reward → slime trains via one `generate` hook. **CPU↔GPU is one env var.**

One image, N sessions, one hook: AgentCore is the isolated per-episode sandbox; slime does the learning.

Section 7

References

References — papers, models & tech reports

- **[1] Agent Lightning** — harnessed agentic RL ·https://arxiv.org/abs/2508.03680
- **[2] DeepSeek-R1** ·https://arxiv.org/abs/2501.12948
- **[3] Qwen3 Technical Report** ·https://arxiv.org/abs/2505.09388
- **[4] Llama-Nemotron** ·https://arxiv.org/abs/2505.00949
- **[5] Kimi K2** ·https://arxiv.org/abs/2507.20534
- **[7] KernelBench** · https://huggingface.co/datasets/ScalingIntelligence/KernelBench
- **[8] MBPP** ·https://arxiv.org/abs/2108.07732
- **[9] slime** — the trainer ·https://github.com/THUDM/slime ·docs https://thudm.github.io/slime/
- **[27] SGLang / RadixAttention** ·https://arxiv.org/abs/2312.07104 ·cache-aware router https://lmsys.org/blog/2024-12-04-sglang-v0-4/

References — RL algorithms & systems (further reading)

- **[13] GRPO (DeepSeekMath)** ·https://arxiv.org/abs/2402.03300
- **[14] DAPO** ·https://arxiv.org/abs/2503.14476
- **[15] GSPO** ·https://arxiv.org/abs/2507.18071
- **[18] AReaL** — asynchronous RL system ·https://arxiv.org/abs/2505.24298
- **[19] HybridFlow / veRL** ·https://arxiv.org/abs/2409.19256
- **[22] Truncated Importance Sampling** — train/inference mismatch · https://fengyao.notion.site/off-policy-rl
- **[28] APRIL** — active partial rollouts ·https://arxiv.org/abs/2509.18521

References — AgentCore docs, SDK & CLI

- **[D2] Host with Runtime** — isolation, compute types · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html
- **[D3] Runtime — how it works** — idle 15 m / max 8 h · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html
- **[D5] Instances (GPU)** ·https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html
- **[D8] InvokeAgentRuntime** ·https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html
- **[D9] CreateAgentRuntime** ·https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html
- **[D13] Observability** — vCPU / GB-hour metrics ·https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html
- **[D15] Python SDK** — `@app.entrypoint` · https://github.com/aws/bedrock-agentcore-sdk-python

References — AgentCore docs, SDK & CLI (cont.)

- **[D16] AgentCore CLI** — `@aws/agentcore` · https://www.npmjs.com/package/@aws/agentcore
- **[D17] Runtime service contract** — :8080, /invocations, /ping · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html